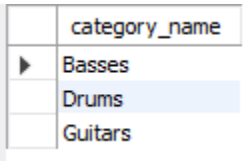
Applied Database Technologies
Lab 5: Subqueries
Luke Colehower

Chapter 7: How to code subqueries

1. Write a SELECT statement that returns the same result set as this SELECT statement, but don't use a join. Instead, use a subquery in a WHERE clause that uses the IN keyword.

```
SELECT DISTINCT category_name
FROM categories c JOIN products p ON c.category_id = p.category_id
ORDER BY category_name DESC
```

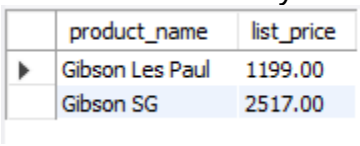


category_name
Basses
Drums
Guitars

```
SELECT DISTINCT category_name
FROM categories c
WHERE category_id IN
(
SELECT category_id
FROM products
);
```

2. Write a SELECT statement that answers this question: Which products have a list price that's greater than the average list price for all products?

   Return the product_name and list_price columns for each product.
   Sort the result set by the list_price column in ascending sequence.



product_name	list_price
Gibson Les Paul	1199.00
Gibson SG	2517.00

```
SELECT product_name, list_price
FROM products
WHERE list_price >
(
SELECT AVG(list_price)
FROM products
)
ORDER BY list_price;
```

3. Write a SELECT statement that returns the category_name column from the categories table.

   Return one row for each category that has never been assigned to any product in

the Products table. To do that, use a subquery introduced with the NOT EXISTS operator.

	category_name
▶	keyboards

```
SELECT category_name
FROM categories c
WHERE c.category_id NOT IN
(
SELECT p.category_id
FROM products p
);
```

4. Write a SELECT statement that returns three columns: email_address, order_id, and the order total for each customer. To do this, you can group the result set by the email_address and order_id columns. *Hint:, you must calculate the order total from the difference of the item_price and discount amount multiplied by the quantity columns in the order_items table.*

	email_address	order_id	order_total
▶	allan.sherwood@yahoo.com	1	839.30
	allan.sherwood@yahoo.com	3	1461.31
	barryz@gmail.com	2	303.79
	christineb@solarone.com	4	1678.60
	david.goldstein@hotmail.com	5	299.00
	david.goldstein@hotmail.com	9	489.30
	erinv@gmail.com	6	299.00
	frankwilson@sbcglobal.net	7	1539.28
	gary_hernandez@yahoo.com	8	679.99

```
SELECT s.email_address, s.order_id, s.order_total
FROM (
   SELECT c.email_address, oi.order_id, SUM((oi.item_price -
oi.discount_amount) * oi.quantity) AS order_total
   FROM order_items oi
```

```
    JOIN orders o USING (order_id)

    JOIN customers c USING (customer_id)

    GROUP BY c.email_address, oi.order_id

) AS s;
```

Write a second SELECT statement that uses the first SELECT statement in its FROM clause. The main query should return two columns: the customer's email address and the largest order for that customer. To do this, you can group the result set by the email_address. Sort the result set by the largest order in descending sequence.

email_address	largest_order
erinv@gmail.com	299.00
barryz@gmail.com	303.79
david.goldstein@hotmail.com	489.30
gary_hernandez@yahoo.com	679.99
allan.sherwood@yahoo.com	1461.31
frankwilson@sbcglobal.net	1539.28
christineb@solarone.com	1678.60

```
SELECT c.email_address, MAX(s.largest_order) AS largest_order

FROM (

    SELECT o.customer_id, SUM((oi.item_price - oi.discount_amount) *
oi.quantity) AS largest_order

    FROM order_items oi

    JOIN orders o USING (order_id)

    GROUP BY o.customer_id

) AS s

JOIN customers c ON c.customer_id = s.customer_id

GROUP BY c.email_address

ORDER BY largest_order;
```

5. Using a correlated subquery, write a SELECT statement that returns the name and discount percent of each product that has a unique discount percent. In other words, don't include products that have the same discount percent as another product.

Sort the result set by the product_name column.

product_name	discount_percent
Gibson SG	52.00
Yamaha FG700S	38.00
Washburn D10S	0.00
Rodriguez Caballero 11	39.00
Hofner Icon	25.00
Tama 5-Piece Drum Set with Cymbals	15.00

```
SELECT product_name, discount_percent
FROM products p
WHERE NOT EXISTS
(
SELECT 1
FROM products p2
WHERE p.discount_percent = p2.discount_percent AND p.product_id !=
p2.product_id
)
ORDER BY product_name;
```

6. Use a correlated subquery to return one row per customer, representing the customer's oldest order (the one with the earliest date). Each row should include these three columns: email_address, order_id, and order_date.

   Sort the result set by the order_date and order_id columns in descending order.

email_address	order_id	oldest_order
allan.sherwood@yahoo.com	1	2018-03-28 09:40:28
barryz@gmail.com	2	2018-03-28 11:23:20
christineb@solarone.com	4	2018-03-30 15:22:31
david.goldstein@hotmail.com	5	2018-03-31 05:43:11
erinv@gmail.com	6	2018-03-31 18:37:22
frankwilson@sbcglobal.net	7	2018-04-01 23:11:12
gary_hernandez@yahoo.com	8	2018-04-02 11:26:38

```
SELECT c.email_address, s.order_id, s.order_date

FROM (

   SELECT o.customer_id, o.order_id, o.order_date

   FROM orders o

   WHERE o.order_date = (

      SELECT MIN(o2.order_date)

      FROM orders o2
```

```
        WHERE o2.customer_id = o.customer_id
    )
) AS s
JOIN customers c ON c.customer_id = s.customer_id
ORDER BY s.order_date, s.order_id;
```